

PlaCo — Kinematics Example

6axis_targets.py (Tablet Annotation Edition)

Repository: <https://github.com/rhoban/placo-examples>

File: kinematics/6axis_targets.py

Library: PlaCo (Rhoban) — Task-space Inverse Kinematics QP solver, built on Pinocchio

Content: PlaCo IK example: a 6-axis arm continuously solving inverse kinematics to reach random end-effector targets, with joint & velocity limits enforced.

Design: Large Typography (11.5pt), Continuous Line Numbering, Function-by-Function Pagination

Table of Contents

1. Imports, Docstring, Robot / Solver Setup & update_target()

Lines 1-49 (49 lines)

2. Main Loop: Real-time IK Solve & Visualization Update

Lines 50-70 (21 lines)

Total: 70 lines across 2 sections.

What is PlaCo (= Planning and Control)

- C++로 작성, Python 바인딩
- Pinocchio 라이브러리 위에 얹혀 있음
- QP 솔버로 최적화
- IK 및 ID 계산 라이브러리

pinocchio

로봇 기구학/동역학 계산 도구 모음
⇒ FK 계산, Jacobian 행렬 계산, 동역학 계산

FK: 관절 각도 → EE 위치
(각 관절의 변환 행렬을 순서대로 곱하기)

IK: 목표 위치 → 관절 각도
· 답이 여러개 또는 없음
· 관절의 한계, 충돌 회피 등 제약조건
· QP(최적화) 문제로 풀어야 함

ID: 목표 힘/가속도 → 필요한 토크
· 마찰, 관성, 다른 관절과 상호작용 고려
· 똑같이 최적화(QP)/제약조건

+ PlaCo는 시뮬레이터 X, 계산 라이브러리 O

⇒ 그림 시각화는? = `placo_utils.visualization`
(부가 패키지)

What is QP = Quadratic Programming, 이차계획법

목표: 오차(목표 위치 - 현재 위치) 최소화
⇒ 오차의 제곱을 최소화

최소화: $\| \text{목표 위치} - \text{현재 위치} \|^2$ ← 오차제곱 (이차식!)

조건: 관절 최소 ≤ ≤ 관절 최대 ← 관절 한계

$\frac{d\theta}{dt} \leq \text{최대 각속도}$ ← 속도 제한

: (제한 조건은 더 많음)

Why PlaCo?

1. Ikpy	순수 파이썬, 가볍고 설치 쉬움	제약 조건 처리 느림
2. PyKDL	오래됨, ROS 생태계 강함	설치 까다로움
3. IK Fast	매우 빠름	로봇마다 사전 코드 필요(범용성↓)
4. pyroki	GPU 가속, 미분 가능	의존성 무거움(JAX)
5. PlaCo	lerobot에 이미 사용중 C++ 코어로 빠름	

+ lerobot도 처음에는 범용 최적화 라이브러리(scipy)로 IK를 구현 ⇒ PlaCo로 바꿈

모델명 irb6620 ← ABB의 산업용 로봇팔

```

1  import pinocchio
2  import placio
3  import numpy as np
4  import pinocchio as pin
5  from ischedule import schedule, run_loop
6  from placio_utils.visualization import robot_viz, robot_frame_viz, frame_viz
7
8  """
9  6axis robot reaching random targets.
10 - Velocity and joint limits are enforced
11 (Depending on the robot configuration, the target may not be reachable)
12 """
13
14 # Loading the robot
15 robot = placio.RobotWrapper("../models/6axis", placio.Flags.ignore_collisions)
16
17 # Creating the solver
18 solver = placio.KinematicsSolver(robot)
19 solver.mask_fbase(True)
20
21 effector_task = solver.add_frame_task("effector", np.eye(4))
22 effector_task.configure("effector", "soft", 1.0, 1.0)
23
24 # Enable velocity limits
25 solver.enable_velocity_limits(True)
26
27 viz = robot_viz(robot)
28
29 def update_target():
30     global effector_task
31     # Choose a random target
32     # 회전 = r = pin.exp6(np.array([0.0, 0.0, 0.0, np.random.uniform([-1.0] * 3, [1.0] * 3)]))
33     # 라포 = t = pin.exp6(np.array([*np.random.uniform([1.0, -1.0, 0.5], [1.5, 1.0, 1.2]), 0.0, 0.0, 0.0]))
34     effector_task.T_world_frame = np.array(t * r)
35
36     t = 0
37     last_target = 0
38     dt = 0.01
39     solver.dt = dt
40     update_target()

```

벡터, 행렬 계산 용

시작화용

6축 모델 정의 및 3D 형상 파일

IK 솔버 객체 생성

ID는 placio.DynamicsSolver

고정된 로봇 (fbase) folating base: 로봇의 몸통이 공중에 떠서 움직이는가? (록어노이드 처럼)

요구사항 추가

대상 프레임

초기 목표 (pose)

4x4 단위행렬

Why? 4x4

6DoF = 회전 3축 + 위치 3축

x, y, z

회전 3축이 가르키고 있는 방향

3차원 이므로 x, y, z 성분 = 3개

frame 가중치

위치(x, y, z) 맞추는 중요도

orientation 가중치

라포: EE와 목표 위치 간 오차를 최대한 줄이되, 0은 강제 X

hard: EE와 목표 위치 무조건 같게 (오차 허용 X)

단위 rad (1rad = 57.3°)

하한, 상한 행렬 사이 랜덤 행렬

단위 m

회전 = r = pin.exp6(np.array([0.0, 0.0, 0.0, np.random.uniform([-1.0] * 3, [1.0] * 3)]))

라포 = t = pin.exp6(np.array([*np.random.uniform([1.0, -1.0, 0.5], [1.5, 1.0, 1.2]), 0.0, 0.0, 0.0]))

→ [라포 3축, 회전 3축] → pin.exp6 → 4x4 행렬

effector_task.T_world_frame = np.array(t * r)

모구서

목표 위치

T_a_b ⇒ T: 변환행렬

a 기준으로 본 b의 pose(위치)

회전 시작(표)

마지막으로 목표 바꾼 시간

한 스텝의 시간 간격(주기)

솔버에게 주기 알려주기

랜덤한 목표 지정

각속도 = $\frac{\Delta \text{각도}}{\text{시간}}$

solver는 각속도 제한 조건 지키기 위해 시간 주기가 필요

4행 문제 의의 (심화)

3x4 행렬 끼리 못 곱함

⇒ 4x4로 변환

4번째 라포가 1=위치(점)

0=벡터

상한

하한

→ [-1.0, -1.0, -1.0] 리스트의 반복 의미

나도 원 소이면지 모르겠다

memo / notes

../models/6axis 구조

robot.urdf
 irb6620_... 1.stl ~ 7.stl
 irb6620_... 1.part ~ 7.part
 config.json
 robot.png

로봇 구조 핵심 XML 파일
 각 링크(뼈) 3D 메쉬
 각 링크 Onshape 부품 데이터
 설정 파일
 미리보기 이미지

→ 웹에서 바로 쓰는 클라우드 기반 3D CAD

```

50 @schedule(interval=dt) ← 데코레이션 (자세한 설명은 아래)
51 def loop():
52     global t, last_target ← 전역 변수
53     t += dt    주기 만큼 계속 더하기
54
55     if t > last_target + 3:    3초가 지나면
56
57         # Updating the target every 3 seconds
58         last_target = t    현재시간 → 마지막으로 목표 바뀐 시간
59         update_target()    새로운 랜덤 목표 위치 설정
60
61     # Solving the IK
62     solver.solve(True)    IK 계산하여 robot.state.q (관절 각도) 갱신
63     robot.update_kinematics()    robot.state.q로 FK 계산 → solver가 EE 위치를 알아야 함
64
65     # Displaying the robot, effector and target
66     viz.display(robot.state.q)
67     robot_frame_viz(robot, "effector")    EE의 좌표축 표시
68     frame_viz("target", effector_task.T_world_frame)    목표 좌리의 좌표축 표시
69
70 run_loop()    loop()를 dt로 간격으로 반복

```

→ False면 갱신X, 계산 결과만 반환

→ 위에서 update_target()

↑ ischedule 라이브러리에 정의된 함수

memo / notes

제어 주기 타이머 { phone-to-sol00 (lerobot)
ischedule (위에 run-loop)

What is 데코레이션?

함수 A { def A():
...
함수 B { def B(func):
...
return func
return func
분문 { A = B(A, dt)

→

def B(func):
...
return func
@B
def A():
...
return func

lerobot
1 작업 실행 → 2 시간 기준 설정 (t0)
3 주기 맞추기
자세한 코드는 해당 예제 분석에

runloop
시간 기준 설정 (t0), 주기: P
t0+P | 작업 실행
t0+2P | 작업 실행
t0+3P | 작업 실행

Why use 데코레이션 나도 모르겠어... 살펴봐

쓰는 이유 : 같은 기능을 여러 함수에 붙일 때
코드가 깔끔해짐
원본 함수 안 건드림

안 쓰는 이유 : 뒤에서 뭐가 일어나는지 안 보임
초보자에게 어렵고 디버깅 어려움
일반 로직로 똑같이 구현됨

필자의 아둔한 시야로는 아직 데코 용법을 이해하지 못했고,
그냥 일반 로직으로 구현하는 것이 이해가 쉽다...

뭔 타이머?

lerobot의 경우 작업 시작 후 시간 측정.
작업이 끝나고 다음 작업으로 넘어갈 때
걸리는 시간만큼 누적 오차 발생!
매 작업마다 타이머 리셋

runloop은 작업 시작전 시간 기준 설정
작업 외부에서 시간 측정
⇒ 오차 최소화

처음 타이머로 끝까지

그럼 lerobot 코드는 어떻게 할까?

(개인의 미숙한 관점입니다)

- 작업 자체의 시간이 중요한 때
- 코드 구현이 쉽다